

RECURSION

1. What is Recursion?

- Recursion is a technique where a function calls itself to solve a smaller instance of the same problem.
- Every recursive solution has two important parts:
 1. **Base Case** - stopping condition that ends the recursion.
 2. **Recursive Case** - reduces the problem to a smaller instance.

★ Key Idea:

Break the problem into smaller pieces of the same kind.

2. General Structure

```
function recurse(parameters):
```

```
    if base case:
```

```
        return answer
```

```
    else:
```

```
        do some work
```

```
        return recurse(smaller parameters)
```

} → Base Case

} → Recursive Case

3. Examples

• Factorial (n!)

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \times (n-1)! & \text{if } n > 0 \end{cases}$$

Code:

```
factorial(n):
```

```
    if n == 0:
```

```
        return 1
```

```
    else:
```

```
        return n * factorial(n-1)
```

• Fibonacci (n)

$$F(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

Code:

```
fibonacci(n):
```

```
    if n == 0:
```

```
        return 0
```

```
    elif n == 1:
```

```
        return 1
```

```
    else:
```

```
        return fibonacci(n-1) + fibonacci(n-2)
```

4. Things to Remember

- Always define a base case, or the recursion will never stop.
- Each recursive call should move the problem closer to the base case.
- Recursion can use more memory due to the call stack.
- Some problems can be solved using iteration instead (may be more efficient).

★ Tip:

Visualize the calls as a tree or stack to understand the flow.

TWO POINTER APPROACH

1. IDEA

- Use two pointers (usually i and j) to traverse a data structure.
- Pointers move toward each other or in the same direction to solve the problem efficiently,
- Helps to reduce time complexity, often from $O(n^2)$ to $O(n)$.

2. WHEN TO USE?

- Array / String problems
- Searching for a pair / target
- Checking palindrome
- Removing duplicates in sorted array
- Partitioning or rearranging elements

3. COMMON PATTERNS

1) Opposite Direction (Toward Each Other)

i starts at beginning, j starts at end.
Move i forward, j backward based on condition.

2) Same Direction

Both pointers move from left to right.
(e.g., removing duplicates)

3) Fast & Slow Pointers

slow moves by 1 step, fast moves by 2 steps.
(e.g., detecting cycle in linked list)

4. COMPLEXITY

- Time Complexity: $O(n)$
- Space Complexity: $O(1)$

5. ADVANTAGES

- Efficient (reduces time complexity)
- Simple and easy to implement
- Works well on sorted data

Example: Pair with Target Sum

Given sorted array 'arr' and target 'T',
find a pair whose sum is T.

Steps:

1. $i = 0$ (start), $j = n - 1$ (end)
2. while $i < j$:
 - if $arr[i] + arr[j] == T \rightarrow$ found
 - if $sum < T \rightarrow i++$
 - if $sum > T \rightarrow j--$
3. If loop ends $\rightarrow$ no pair found

TIPS

- Make sure the data is sorted if needed.
- Be careful with boundary conditions ($i < j$).
- Think about how pointers should move based on the condition.